Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 10 2026

The Byte Strip

Eight cards. No pens, no arithmetic, no subtraction.

Start here. Cut the eight cards off the bottom of this sheet. Lay them on the mat below, **biggest on the left**, one card per square.

The back of every card is blank, and that is the point. A card **face up** means that place value is switched **on** — a `1`. A card turned **blank side up** means **off** — a `0`. The square underneath tells you which card it is, so you never lose your place.

Move 1 Show me a number. (denary → binary)
Start with every card blank side up. Mr. Rawson calls a number. Turn over the **biggest card that fits**. Then the next biggest that fits. Keep going until the cards showing add up to exactly his number. **Never turn a card that would take you over.** Read the row left to right — that is the binary.

Move 2 Read me a number. (binary → denary)
Mr. Rawson calls out eight digits. Set the cards to match, then **add up what is face up**. Same cards, run backwards.

Move 3 Split the strip. Slide the four right-hand cards away from the four left-hand cards. **Each group of four is a nibble.** Go to page 2.

Move 4 Build the strip you keep. Fill in the page 2 table with your own cards. **Keep it** — it is your reference for the rest of this unit, for the **quiz on Thu 17 Sept** and for the **test on Tue 6 Oct**.

THE BYTE MAT · one card per square

128	**64**	**32**	**16**	**8**	**4**	**2**	**1**

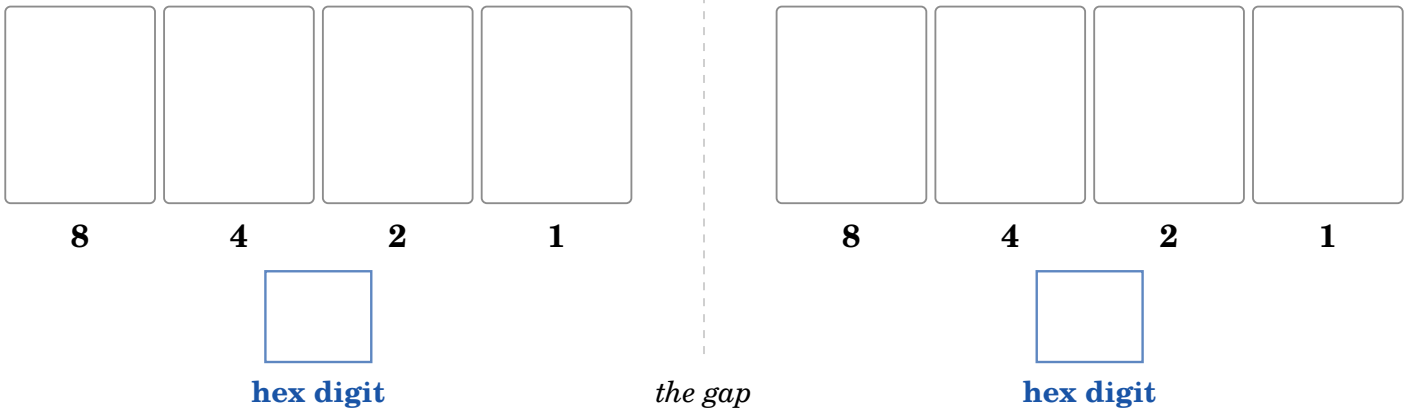
✂ **Cut along the dashed line, then cut the eight cards apart.**

128	**64**	**32**	**16**	**8**	**4**	**2**	**1**

Move 3 · Split the strip

Name: _____

Put four cards on each mat below and **read each mat on its own**. The left-hand four were 128, 64, 32, 16 on the byte mat — on their own mat they read `8 4 2 1`, exactly like the right-hand four. **A nibble is a nibble wherever it sits**, and each one can only show **0 to 15**.



The one number that shows the whole idea

`0011` `0100`
3 **4**

Read it as **two nibbles**: `0011` = 3, `0100` = 4 → **34** in hexadecimal

Read it as **one byte**: $32 + 16 + 4$ → **52** in denary

These are not two different numbers. One pattern of eight cards, written down two ways: **34** in hex **is 52** in denary. Check it: $3 \times 16 + 4 = 52$.

Hexadecimal is just where you put the gap. Nothing about the cards changed.

Your turn. Set your cards to `1101 1110`.

a) In hexadecimal it is _____

b) In denary it is _____

c) Check: ____×16+____ = _____

Move 4 · Your reference strip — build it, keep it

Use **one** nibble mat. Make every number from 0 to 15 with your four cards, and write down what you see. Two rows are done for you.

Denary	8	4	2	1	Hex
0	0	0	0	0	0
1					
2					
3					
4					
5					
6					
7					

Denary	8	4	2	1	Hex
8					
9					
10	1	0	1	0	A
11					
12					
13					
14					
15					

Four cards run out at 15 — which is the whole reason hexadecimal needs letters. Sixteen values, only ten digits.

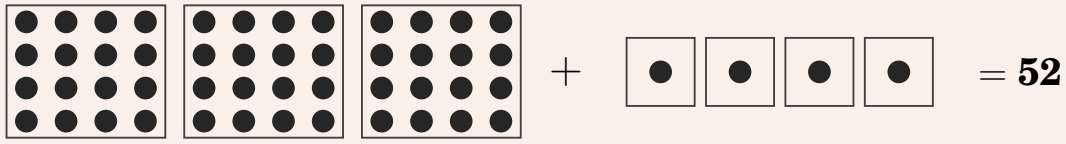
Now turn over — what is a nibble actually worth?

Move 5 · The sixteen-card

On page 2 you split `0011 0100` into two nibbles and read **3** and **4**. **But the left nibble was never just 3.** It was **3 sixteens**. Cut out the sixteen cards on the next page, take a set of singles, and come back.

In a hex pair, the left digit counts *cards* and the right digit counts *dots*.

34 — a number you have already met



3 cards = 48 **4 singles** **= 52**

Same number, third time. On page 2 this byte was `0011 0100`, and $32 + 16 + 4 = 52$. In hex it is `34`. On the desk it is three cards and four singles. **Nothing has changed but how you are looking at it.**

Your turn. Build each one, then check the arithmetic.

	Cards × 16	**Singles**	**Total in denary**
`4B`			
`7C`			
`DE`			

You met `DE` on page 2, where the cards said 222. If the desk and the byte strip disagree, one of them is wrong — and finding out which is the useful part.

Count out sixteen singles.

Now put them beside **one card**. They are worth exactly the same. **Sixteen singles *are* one card** — so swap them, and the right-hand digit drops to nought while the left-hand one goes up by one.

That swap is the only thing place value has ever meant.

Then stop using them.

You can build *every* byte there is — `FF` is 15 cards and 15 singles. But laying `DE` out takes a minute, and working it out takes five seconds: $13 \times 16 = 208$, plus **14**, is **222**.

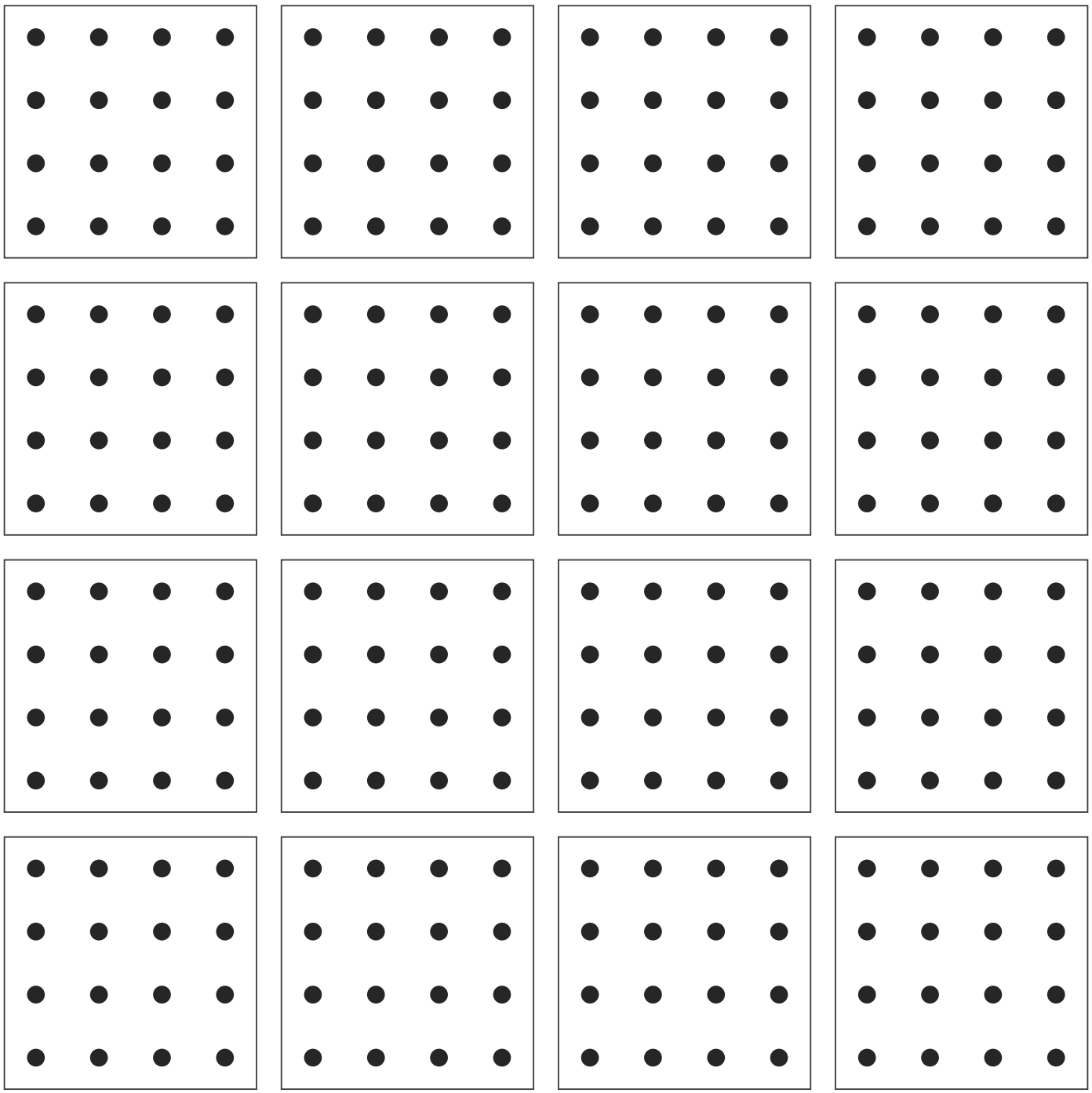
The cards are how you learn to trust the arithmetic. Then you leave them behind.

Now put all sixteen cards on the desk at once. That is $16 \times 16 = \mathbf{256}$ dots — but the biggest number a byte can hold is **255**, because `FF` is **15 cards and 15 singles** $= 240+15 = 255$. **So the sixteenth card is one too many.** A byte counts **0 to 255**: 256 different values, and you have just laid every one of them on the table.

And next: a colour code is three byte strips side by side — each channel gets its own sixteen cards.

The sixteen-cards

Sixteen cards, sixteen dots on each. Cut down the white gaps — three cuts down, three across.

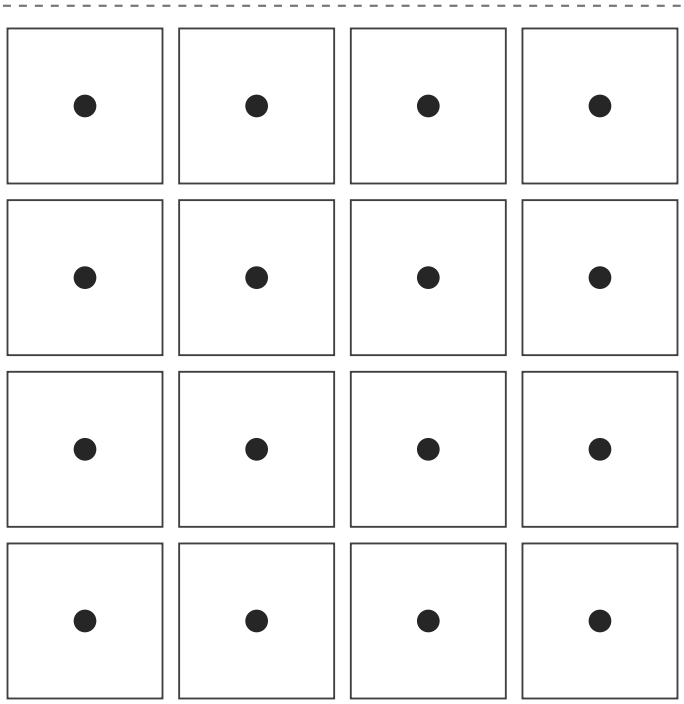


Sixteen cards is $16 \times 16 = 256$ *dots — one more than the biggest number a single byte can hold.*

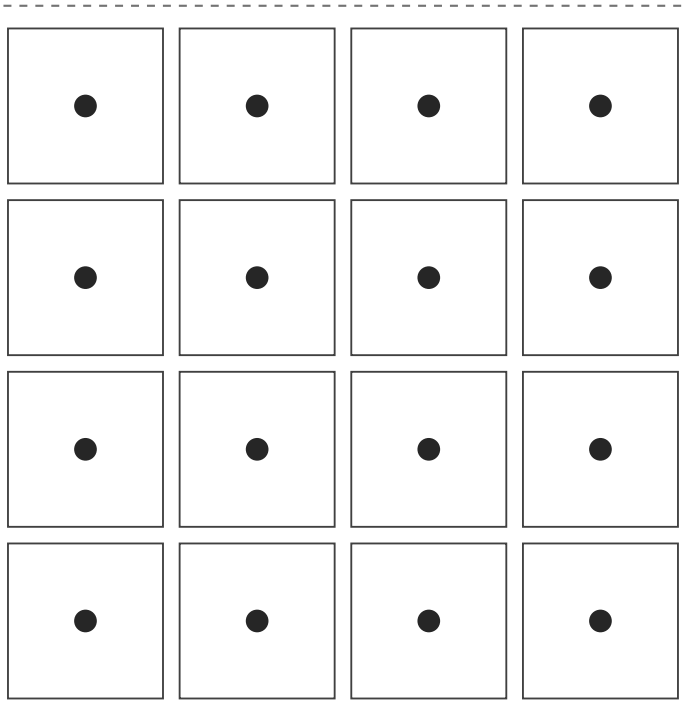
The singles

Four sets of sixteen. Cut along the dashed lines first, then cut each set into its tiles.

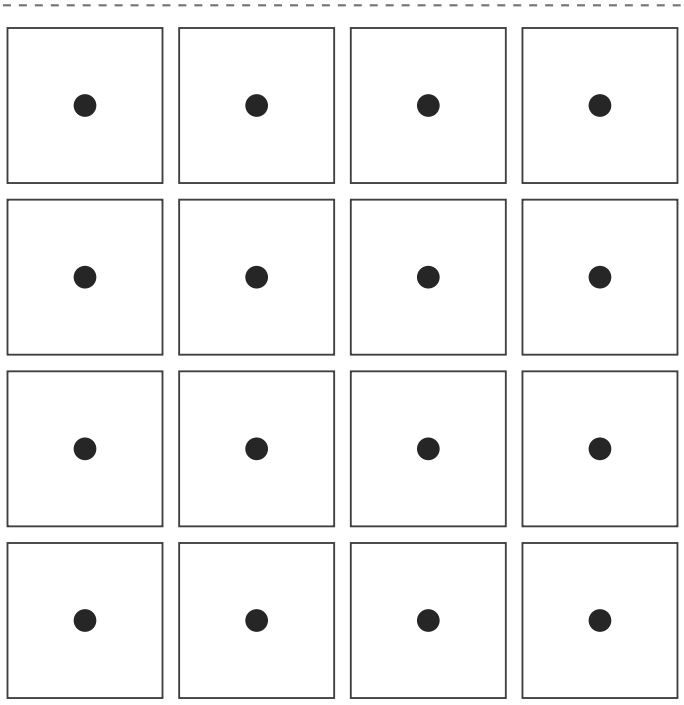
ONE SET OF 16



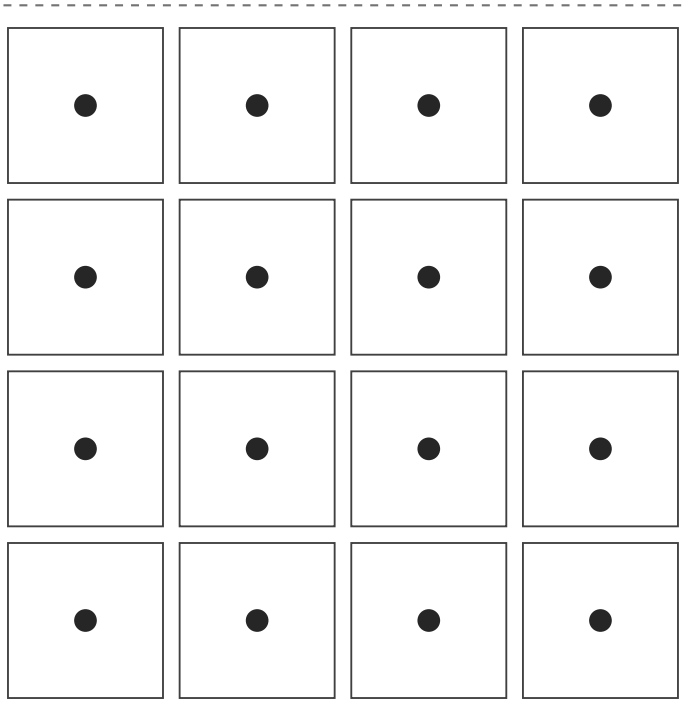
ONE SET OF 16



ONE SET OF 16



ONE SET OF 16



Each dashed block is one set of sixteen — enough for the right-hand hex digit, which never goes past 15. Sixteen of them would be a whole card, which is exactly the point of Move 5.